AIAC LAB -12

Lab Assignment: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Name : G.Sreeja
Roll No : 2503a52692
Batch No : 24

Question 1: Insertion Sort – Employee Salary Management

A company maintains employee salary records. The system needs to
arrange salaries in ascending order.

(a) Program: Insertion Sort

```
def insertion_sort(a):
    for i in range(1,len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]  # shift larger element right
            j -= 1
        a[j + 1] = key       # insert the key
        print(f"Iteration{i}: key = {key} -> {a}")

salaries = [float(x) for x in input("Enter salaries: ").split()]
print("Original:", salaries)
insertion_sort(salaries)
print("Sorted:", salaries)
```

(b) Role of the key element and shifting operations

- The key is the element currently being inserted (the element at positioni). It is saved in a variable first, because shifting overwrites its original position.
- The part of the array to the left of the key is already sorted.
- Shifting: every salary in the sorted part that is greater than the key is moved one position to the right. This opens a gap in the correct place.
- The key is then inserted into that gap, so the sorted part grows by one element after every iteration.

(c) Array displayed after each iteration (sample run)

Enter salaries: 45000 30000 52000 28000 39000
Original: [45000.0, 30000.0, 52000.0, 28000.0, 39000.0]
Iteration 1: key = 30000.0 -> [30000.0, 45000.0, 52000.0, 28000.0, 39000.0]
Iteration 2: key = 52000.0 -> [30000.0, 45000.0, 52000.0, 28000.0, 39000.0]
Iteration 3: key = 28000.0 -> [28000.0, 30000.0, 45000.0, 52000.0, 39000.0]
Iteration 4: key = 39000.0 -> [28000.0, 30000.0, 39000.0, 45000.0, 52000.0]
Sorted: [28000.0, 30000.0, 39000.0, 45000.0, 52000.0]

(d) Efficiency for nearly sorted data

Case	Time complexity	Reason
Best	O(n)	Data already sorted: each key needs only on comparison and no shifting

Average	$O(n^2)$	About half of the sorted part is shifted for each key

Space complexity: O(1), because sorting is done in place.

For nearly sorted data, only a few elements are out of place, so the inner while loop runs very few times for each key. The running time therefore approaches O(n), which makes Insertion Sort efficient for nearly sorted data.

Question 2: Merge Sort – Hospital Patient Records
A hospital maintains thousands of patient records containing Patient ID, Patient Name, and Medical Bill Amount.

a) Program: Merge Sort on medical bill amount

```
def merge_sort(r):
    if len(r) <= 1:
        return r
    mid =len(r) // 2
    print("Divide:", [x[2] for x in r], "->", [x[2] for x in r[:mid]], [x[2] for x in r[mid:]])
    left, right =merge_sort(r[:mid]),merge_sort(r[mid:])
    merged = []
    while left and right:
        merged.append(left.pop(0) if left[0][2] <= right[0][2] else right.pop(0))
    merged += left + right
    print("Merge: ", [x[2] for x in merged])
    return merged
```

```
n = int(input("Number of patients: "))
records = []
for _ in range(n):
    pid, name, bill = input("Patient ID, Name, Bill: ").split()
    records.append((pid, name, float(bill)))
for pid, name, bill in merge_sort(records):
    print(pid, name, bill)
```

(b) The divide-and-conquer process

- Divide:the list of records is split into a left half and a right half, again and again, until every part has exactly one record.
- Conquer:a part with one record is already sorted.
- Combine:two sorted parts are merged by repeatedly comparing their first records and taking the one with the smaller bill amount. Whole records are moved, so the Patient ID and Name stay with their bill.

(c) Splitting and merging operations (sample run)

```
Number of patients: 5
Patient ID, Name, Bill: P101 Ravi 4500
Patient ID, Name, Bill: P102 Asha 1200
Patient ID, Name, Bill: P103 Meena 8800
Patient ID, Name, Bill: P104 Kiran 3000
Patient ID, Name, Bill: P105 Sana 6100
Divide: [4500.0, 1200.0, 8800.0, 3000.0, 6100.0] -> [4500.0, 1200.0]
[8800.0, 3000.0, 6100.0]
Divide: [4500.0, 1200.0] -> [4500.0] [1200.0]
```

Merge: [1200.0, 4500.0]
Divide: [8800.0, 3000.0, 6100.0] -> [8800.0] [3000.0, 6100.0]
Divide: [3000.0, 6100.0] -> [3000.0] [6100.0]
Merge: [3000.0, 6100.0]
Merge: [3000.0, 6100.0, 8800.0]
Merge: [1200.0, 3000.0, 4500.0, 6100.0, 8800.0]
P102 Asha 1200.0
P104 Kiran 3000.0
P101 Ravi 4500.0
P105 Sana 6100.0
P103 Meena 8800.0

(d) Time and space complexity

Time: the list is halved about log n times (log n levels), and merging all parts on each level costs about n comparisons. So the total is O(n log n) in every case, which is why Merge Sort suits thousands of patient records.

- Space: O(n), because temporary lists are created while merging.

Question 3: Selection Sort – Product Price Management

An online shopping platform maintains a list of product prices.

(a) Program: Selection Sort (ascending)

```
prices = [float(x) for x in input("Enter prices: ").split()]
for i in range(len(prices) - 1):
 m = min(range(i, len(prices)), key=lambda k: prices[k])  # index of minimum
 prices[i], prices[m] = prices[m], prices[i]
 print(f"Pass {i + 1}: minimum = {prices[i]} -> {prices}")
print("Sorted:", prices)
```

(b) Minimum element in every pass and array after each pass (sample run)

Enter prices: 64 25 12 22 11
Pass 1: minimum = 11.0 -> [11.0, 25.0, 12.0, 22.0, 64.0]
Pass 2: minimum = 12.0 -> [11.0, 12.0, 25.0, 22.0, 64.0]
Pass 3: minimum = 22.0 -> [11.0, 12.0, 22.0, 25.0, 64.0]
Pass 4: minimum = 25.0 -> [11.0, 12.0, 22.0, 25.0, 64.0]
Sorted: [11.0, 12.0, 22.0, 25.0, 64.0]

Pass	Minimum found	Array after the pass
1	11	[11, 25, 12, 22, 64]
2	12	[11, 12, 25, 22, 64]
3	22	[11, 12, 22, 25, 64]
4	25	[11, 12, 22, 25, 64]

In each pass the program scans the unsorted part to find the smallest price, then swaps it with the first position of the unsorted part. The sorted section grows by one position every pass.

c) Time and space complexity

Case	Time complexity
Best	$O(n^2)$
Average	$O(n^2)$

Worst $O(n^2)$

Time: Selection Sort always scans the whole unsorted part to find the minimum, even if the list is already sorted, so it takes about n(n−1)/2 comparisons in every case.

- Space: O(1), because sorting is done in place using only a few extra variables.

Question 4: Bubble Sort – Student Marks Sorting

A university maintains student records containing Roll Number,

Student Name, and Marks.

(a) Program: Bubble Sort (ascending order of marks)

```
n = int(input("Number of students: "))
s = []
for _ in range(n):
 roll, name, marks = input("Roll, Name, Marks: ").split()
 s.append((roll, name, float(marks)))
for p in range(n - 1):
 swapped = False
 for i in range(n - 1 - p):
 if s[i][2] > s[i + 1][2]:
 s[i], s[i + 1] = s[i + 1], s[i]  # swap the whole record
 swapped = True
 print(f"Pass {p + 1}:", [(x[0], x[2]) for x in s])
 if not swapped:
 break
for roll, name, marks in s:
 print(roll, name, marks)
```

(b) Array displayed after each pass (sample run)

Number of students: 5
Roll, Name, Marks: 101 Ravi 75
Roll, Name, Marks: 102 Asha 88
Roll, Name, Marks: 103 Meena 92
Roll, Name, Marks: 104 Kiran 67
Roll, Name, Marks: 105 Sana 81
Pass 1: [('101', 75.0), ('102', 88.0), ('104', 67.0), ('105', 81.0), ('103', 92.0)]
Pass 2: [('101', 75.0), ('104', 67.0), ('105', 81.0), ('102', 88.0), ('103', 92.0)]
Pass 3: [('104', 67.0), ('101', 75.0), ('105', 81.0), ('102', 88.0), ('103', 92.0)]
Pass 4: [('104', 67.0), ('101', 75.0), ('105', 81.0), ('102', 88.0), ('103', 92.0)]
104 Kiran 67.0
101 Ravi 75.0
105 Sana 81.0
102 Asha 88.0
103 Meena 92.0

(b) Array displayed after each pass (sample run)

Number of students: 5
Roll, Name, Marks: 101 Ravi 75
Roll, Name, Marks: 102 Asha 88
Roll, Name, Marks: 103 Meena 92
Roll, Name, Marks: 104 Kiran 67
Roll, Name, Marks: 105 Sana 81
Pass 1: [('101', 75.0), ('102', 88.0), ('104', 67.0), ('105', 81.0), ('103', 92.0)]
Pass 2: [('101', 75.0), ('104', 67.0), ('105', 81.0), ('102', 88.0), ('103', 92.0)]
Pass 3: [('104', 67.0), ('101', 75.0), ('105', 81.0), ('102', 88.0), ('103', 92.0)]
Pass 4: [('104', 67.0), ('101', 75.0), ('105', 81.0), ('102', 88.0), ('103', 92.0)]

104 Kiran 67.0

101 Ravi 75.0

105 Sana 81.0

102 Asha 88.0

103 Meena 92.0

(c) The swapping process

- Bubble Sort compares each pair of neighbouring students by marks.
- If the left student has more marks than the right student, the two are swapped. In Python this is: students[i], students[i+1] = students[i+1], students[i].
- The entire record (roll number, name and marks) is swapped, not only the marks, so every student keeps the correct roll number and name.
- After each pass the largest remaining mark "bubbles" to the end of the unsorted part, so each pass needs one comparison fewer.
- If a pass makes no swaps, the list is already sorted and the program stops early.

Case	Time complexity
Best	O(n), when already sorted (with the early-stop check)
Average	$O(n^2)$
Worst	$O(n^2)$, when in reverse order

Space complexity: O(1), because records are sorted in place.

Question 5: Quick Sort – Examination Result Processing

A university maintains examination results containing Roll Number,

Student Name, and Total Marks.

(a) Program: Quick Sort (descending order of total marks, last element as pivot)

```
def quick_sort(s, low, high):
    if low < high:
        pivot, b = s[high][2], low - 1       # last element is the pivot
        fori in range(low, high):
            if s[i][2] >= pivot:             # higher marks go first (descending)
                b += 1
                s[b], s[] = s[i], s[b]
        s[b + 1], s[high] = s[high], s[b + 1]  # place pivot in final position
        print(f"Pivot{pivot} -> index {b + 1}:", [x[2] for x in s])
        quick_sort(s, low, b)
        quick_sort(s, b + 2, high)

n = int(input("Number of students: "))
s = []
for _ in range(n):
    roll, name, marks = input("Roll, Name, Total marks: ").split()
    s.append((roll, name, float(marks)))
quick_sort(s, 0, n - 1)
for roll, name, marks in s:
    print(roll, name, marks)
```

(b) Pivot selection, partitioning and recursive calls (sample run)

```
Number of students: 5
Roll, Name, Total marks: 101 Ravi 450
Roll, Name, Total marks: 102 Asha 520
Roll, Name, Total marks: 103 Meena 480
Roll, Name, Total marks: 104 Kiran 390
```

Roll, Name, Total marks: 105 Sana 510
Pivot 510.0 -> index 1: [520.0, 510.0, 480.0, 390.0, 450.0]
Pivot 450.0 -> index 3: [520.0, 510.0, 480.0, 450.0, 390.0]
102 Asha 520.0
105 Sana 510.0
103 Meena 480.0
101 Ravi 450.0
104 Kiran 390.0

Number of students: 5
Roll, Name, Total marks: 101 Ravi 450

Roll, Name, Total marks: 102 Asha 520

Roll, Name, Total marks: 103 Meena 480

Roll, Name, Total marks: 104 Kiran 390

Roll, Name, Total marks: 105 Sana 510

Pivot 510.0 -> index 1: [520.0, 510.0, 480.0, 390.0, 450.0]

Pivot 450.0 -> index 3: [520.0, 510.0, 480.0, 450.0, 390.0]

102 Asha 520.0

105 Sana 510.0

103 Meena 480.0

101 Ravi 450.0

104 Kiran 390.0

(c) Explanation of the partitioning process and recursive calls

- The last element of the current section is chosen as the pivot.
- Partitioning: each other record is compared with the pivot. Since the order is descending, a record with marks greater than or equal to the pivot is moved to the front (higher-marks section). Records with fewer marks stay after it.
- The pivot is then swapped into the position just after the higher-marks section. At that point the pivot is in its final sorted position.
- Recursive calls: Quick Sort is called again on the left section (higher marks) and the right section (lower marks). This repeats until a

section has zero or one record, which is already sorted.

- In the sample run, the pivot 510 (roll 105) is placed at index 1. The right section (indexes 2 to 4) is then partitioned with pivot 450.

(d) Best, average and worst-case time complexity

Case	Time complexity	When it happens
Best	O(n log n)	The pivot splits the records into two nearly equal halves every time
Average	O(n log n)	Pivots split the records reasonably well on random data
Worst	$O(n^2)$	The pivot is always the largest or smallest valu (for example, already sorted data with the las element as pivot), so one section is empty eacl time

Space complexity: O(log n) on average for the recursion stack, and O(n) in the worst case.